

Homework 2 (DAC-102 and DAE-101 Computer Architecture 2026)

Mehta Family School of Data Science and Artificial Intelligence, IIT Roorkee

Topics: Floating-point number and RISC-V Instruction Encoding

Submit your work only as a PDF.

1. Represent 28.6989 in single-precision and double-precision floating point notation (you may use online tool for this). [total 2 mark] Also, find the difference between the number (i.e., **28.6989**) and what is stored for both single and double precision-numbers. [total 2 mark]
Try to observe, in which case, is the error (difference) smaller?
2. Show the range (largest and smallest positive normal numbers) of a number format, which has 7 bits of exponent and 17 bits of mantissa and 0 bit of sign. Do the calculation to show the steps. [2 marks]
3. Consider an FP number system with total of 22 bits. It has no sign bit, 11 exponents and remaining mantissa bits. Show the bias (1mark), and largest denormal number (2 mark).
4. In this question, you will use an online converter (such as <https://www.h-schmidt.net/FloatConverter/IEEE754.html> or https://www.binaryconvert.com/convert_float.html), or if these are not available, you can do the conversion manually. We take a number 49.7 and store it in IEEE-754 format. We find its binary equivalent from above website. Now, the leftmost bit is the sign-bit, and then, there are 8 exponent bits. Flip the second bit from left, which is the most significant among exponent bits and then write the decimal value of new number, up to 7 digits after decimal. (Lesson: any change in exponent bit leads to large change in the magnitude of a number.) [2 mark, no partial marking]
5. Let “4+1” show 4 exponent bits and 1 mantissa bits. Fill the below table. [0.5 marks for each answer below]

	4+1	4+2
Number of normal numbers		
Difference between consecutive numbers at the smallest number		
Difference between consecutive numbers at the largest number		
Range written as [Smallest, Largest]		

6. Use encoding tables to encode the following instructions. [5 marks]

(i) add x9, x12, x22
(ii) srli x10, x22, 3
(iii) lbu x10, 22(x12)
(iv) sw x15, 24(x14)
(v) beq x8, x16, 4000

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]					rs1		funct3		rd		opcode		I-type	
imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode		S-type
imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode		B-type
imm[31:12]										rd		opcode		U-type
imm[20 10:1 11 19:12]										rd		opcode		J-type
addi	ADD Immediate					I	0010011	0x0	imm[5:11]=0x00 imm[5:11]=0x00 imm[5:11]=0x20		rd = rs1 + imm			
xori	XOR Immediate					I	0010011	0x4			rd = rs1 ^ imm			
ori	OR Immediate					I	0010011	0x6			rd = rs1 imm			
andi	AND Immediate					I	0010011	0x7			rd = rs1 & imm			
slli	Shift Left Logical Imm					I	0010011	0x1			rd = rs1 << imm[0:4]			
srli	Shift Right Logical Imm					I	0010011	0x5			rd = rs1 >> imm[0:4]			
srai	Shift Right Arith Imm					I	0010011	0x5			rd = rs1 >> imm[0:4]			
slti	Set Less Than Imm					I	0010011	0x2			rd = (rs1 < imm)?1:0			
sltiu	Set Less Than Imm (U)					I	0010011	0x3			rd = (rs1 < imm)?1:0			
lb	Load Byte					I	0000011	0x0			rd = M[rs1+imm][0:7]			
lh	Load Half					I	0000011	0x1			rd = M[rs1+imm][0:15]			
lw	Load Word					I	0000011	0x2			rd = M[rs1+imm][0:31]			
lbu	Load Byte (U)					I	0000011	0x4			rd = M[rs1+imm][0:7]			
lhu	Load Half (U)					I	0000011	0x5			rd = M[rs1+imm][0:15]			
sb	Store Byte					S	0100011	0x0			M[rs1+imm][0:7] = rs2[0:7]			
sh	Store Half					S	0100011	0x1			M[rs1+imm][0:15] = rs2[0:15]			
sw	Store Word					S	0100011	0x2			M[rs1+imm][0:31] = rs2[0:31]			
beq	Branch ==					B	1100011	0x0			if(rs1 == rs2) PC += imm			
bne	Branch !=					B	1100011	0x1			if(rs1 != rs2) PC += imm			
blt	Branch <					B	1100011	0x4			if(rs1 < rs2) PC += imm			
bge	Branch ≥					B	1100011	0x5			if(rs1 ≥ rs2) PC += imm			
bltu	Branch < (U)					B	1100011	0x6			if(rs1 < rs2) PC += imm			
bgeu	Branch ≥ (U)					B	1100011	0x7			if(rs1 ≥ rs2) PC += imm			